Pandas Cheatsheet

Data Processing and Representations

1. Import Pandas

```
import pandas as pd
```

2. Creating a Series

```
s = pd.Series([10, 20, 30, 40],
    index=['a', 'b', 'c', 'd'])
```

	0
a	10
b	20
c	30
d	40

3. Creating a DataFrame

```
df = pd.DataFrame({'Age': [28, 24, 35, 28],
    'City': ['York', 'Oslo', 'Lima', 'York']},
    index = ['John', 'An', 'Peter', 'Tom'])
```

	Age	City
John	28	York
An	24	Oslo
Peter	35	Lima
Tom	28	York

4. Column Operations: Adding, Multiplying

```
# Adding a new column
df['Salary'] = [50000, 60000, 75000, 55000]
# Compute new column
df['Bonus'] = df['Salary'] * 0.10

# Adding, subtracting, multiplying columns
df['Total'] = df['Salary'] + df['Bonus']
df['Years Until Retirement'] = 65 - df['Age']
```

5. Reading and Writing

```
# Using commas as separator
df = pd.read_csv('data.csv', sep=',')
df.to_csv('output.csv', index=False)
```

6. Accessing Data (Indexing and Slicing)

```
# Accessing rows:
df.iloc[3]          # Access by position
df.loc['Peter']     # Access by index label

# Multiple indices
df.iloc[[2, 3]]
df.loc[['An', 'Peter']]

# Slicing rows
df.iloc[0:3:2]      # Rows 0 to 3 (steps of 2)

# Accessing columns
df['Age']           # Single column -> Series
df[['City', 'Age']] # Multiple columns -> DF

# Column indexing (position-based)
df.iloc[:, [0, 2]]  # Columns at positions 0 and 2
df.iloc[:, 0:2]     # Slice columns by position
```

7. Statistics

```
df['Age'].mean()          df['Age'].min()
df['Age'].median()        df['Age'].max()
df['Age'].sum()           df['Age'].std()

# Getting the index label of an extreme value
df['Age'].idxmax()
df['Age'].idxmin()

df['Age'].count()         # Count non-null values
df['City'].unique()       # Unique elements
df['City'].value_counts() # Element frequency
df['City'].nunique()      # Number of uniques
```

8. Filtering Data

```
# Filter rows based on a single condition.
df[df['Age'] > 30]

# Combine multiple conditions using & and |
filtered_df = df[
    (df['Age'] > 30) & (df['City'] == 'Lima')]
filtered_df = df[
    (df['Age'] < 30) | (df['City'] == 'York')]

# Membership testing
df[df['City'].isin(['York', 'Lima'])]
df[~df['City'].isin(['Oslo'])]   # Negation
```

9. Dropping Data

```
# Drop rows
df = df[~df['City'].eq('Oslo')]  # By mask
df = df.drop('John')             # By index

# Drop columns
df = df.drop(columns=['Age'])    # By name
df = df.drop('Age', axis=1)      # By name
df = df.iloc[:, [1, 3]]          # By index
```

10. Grouping

```
grouped = df.groupby('City')

# Looping over groups
for city, group in grouped:
    print(f"\nCity: {city}")
    print(group)
```

11. Aggregation Functions

```
grouped['Age'].mean()     grouped['Age'].min()
grouped['Age'].median()   grouped['Age'].max()
grouped['Age'].sum()      grouped['Age'].std()

grouped['Age'].count() # Count non-null values
grouped.head(n)        # First n rows per group
grouped.tail(n)        # Last n rows per group
```

12. Sorting Data

```
# Sort by a single column
df.sort_values(by='Age')

# Sort by multiple columns
df.sort_values(by=['Age', 'Salary'],
    ascending=[True, False])
```

13. Map and Replace

```
# One-to-one mapping on a Series
city_to_country = {'York': 'UK', 'Oslo': 'NO',
    'Lima': 'PE'}
df['Country'] = df['City'].map(city_to_country)

# Map with a named function
def age_band(a):
    if a >= 30:
        return '30+'
    return '<30'

df['AgeBand'] = df['Age'].map(age_band)

# Replace specific values (dict or scalar)
df['City'] = df['City'].replace({'York': 'USA',
    'Lima': 'USA'})
df['City'] = df['City'].replace('York', 'USA')
```

14. Combining DataFrames

```
# Combine horizontally
combined = pd.merge(df1, df2, on='City',
    how='left') # 'inner'/'outer'/'left'/'right'

# Combine vertically + avoiding duplicate indexes
combined = pd.concat([df1, df2], ignore_index=True)
```

15. Dtypes & Casting

```
df.dtypes           # column dtypes
df.info()           # non-null counts + dtypes
df['Age'].dtype

# Convert types
df['Age'] = df['Age'].astype('int')
```

16. Index Management

```
# Make one of the columns the index
df = df.set_index('City')

# Reset index to numbered, inserts 'City' back
df = df.reset_index()
```

17. Missing Data

```
# NA count per column
df.isna().sum()

# Drop rows where Age is NA
df = df.dropna(subset=['Age'])

# Replace NA with 0 in one column
df['Age'] = df['Age'].fillna(0)

# Or fill multiple columns
df = df.fillna({'Age': 0, 'City': 'Unknown'})
```

18. Aggregation

```
# Aggregate by different functions
df.groupby('City')['Age'].agg(['mean', 'median',
    'count', 'nunique'])

# First/last per group
df.groupby('City')['Age'].agg(['first', 'last'])
```

19. Iteration with `iterrows()`

Note that using `iterrows()` usually is not the best solution. Instead try to use masking, or `map()`!

```
# Collect rows that meet requirements
old_people_ids = []
for idx, row in df.iterrows():
    if row['Age'] >= 30:
        old_people_ids.append(idx)

# old_people_ids now holds the index labels
```

20. Reshaping: explode, melt, pivot, pivot_table

```
# Explode: list-like -> one row per item
df2 = pd.DataFrame({'Name': ['John','An'],
    'Skills': [['py','sql'], ['r']]})
df2_ex = df2.explode('Skills', ignore_index=True)

# Moving the index into a column for melt
df_unindexed = df.reset_index(names='Name')

# Wide -> long (keep Name and City as indexes)
df_long = df_unindexed.melt(
    id_vars=['Name', 'City'],
    value_vars=['Age', 'Salary'],
    var_name='Var', value_name='Val')

# Long -> wide (requires unique index/column pairs)
df_wide = df_long.pivot(index=['Name', 'City'],
    columns='Var', values='Val')

# If duplicates exist, use pivot_table with agg
pt = pd.pivot_table(df_long, index='City',
    columns='Var', values='Val',
    aggfunc='mean')
```